

实践10：基于PHP的HOI人-物交互运动跟踪



深蓝学院《人形机器人运动控制》课程第7章

课程链接: <https://www.shenlanxueyuan.com/course/864>

1. 作业目标

本次作业要求在 Isaac Lab 中为 Unitree G1 29DoF 的 HOI人-物交互运动跟踪任务接入 RayCaster 地形高度感知, 并完成代码实现、短训练验证、感知结果可视化和策略回放。

本次作业使用的任务 ID 为:

代码块

```
1 Unitree-G1-29dof-Mimic-HOI_terrain-Perceptive-Raycast
```

最终需要打通的链路如下:

代码块

```
1 HOI motion + terrain metadata/URDF
2     ↓
3     G1 motion tracking 环境
4     ↓
5     terrain mjcf_boxes → merged Warp mesh
6     ↓
7     torso_link RayCaster → height scan
8     ↓
9     policy / critic observations
10    ↓
11    PPO
12    ↓
13    checkpoint 与回放结果
```

完成本次作业后, 应能够理解:

- HOI 参考运动、机器人状态与地形资产如何共同组成 motion tracking 环境。
- blind motion tracking 与 perceptive motion tracking 的区别。

- 如何从 `*.terrain.json` 中读取地形 box primitive，并生成可供射线求交的地形 mesh。
- RayCaster 的挂载位置、扫描方向、采样网格与高度观测的含义。
- policy observation 与 critic observation 为什么需要使用不同的噪声配置。
- 如何使用 Isaac Lab Observation Manager 管理多帧历史观测。
- 如何通过 smoke train、RayCaster 可视化、训练日志和 checkpoint 回放验证完整感知运控链路。

2. 实践背景

2.1 论文背景

本实践参考论文 [Perceptive Humanoid Parkour: Chaining Dynamic Human Skills via Motion Matching](#)。论文面向 Unitree G1 人形机器人提出了一套感知式跑酷框架，目标是让机器人利用机载感知完成攀爬、翻越、落地和多障碍连续通过等高动态动作。

论文的完整技术路线主要包含三个阶段：

1. 使用 OmniRetarget 将人类动作重定向为机器人可执行的原子技能。
2. 通过 Motion Matching 将行走/奔跑与短时跑酷技能组合成长时域运动轨迹。
3. 先训练使用特权状态和 height scan 的 motion-tracking expert，再使用 DAgger 与 PPO 将多个 expert 蒸馏为单个深度图感知 student policy。

论文中的完整系统解决的是“动作数据生成—技能组合—专家跟踪—视觉策略蒸馏—真机迁移”问题，而本次课程作业只选择其中与地形感知 motion tracking 相关的一部分进行实践，因此本实践可简称为 HOI (Humanoid-Object Interaction) parkour。

2.2 本实践的复现范围

本实践**不是**对论文的完整复现。为了将任务控制在一次课程作业可完成的范围内，仓库已经提供参考动作、地形、motion tracking 环境、PPO 配置、RayCaster 实现和调试工具，同学**只需要完成感知链路中的三组核心 TODO**，该部分在后面作业会详细介绍。

对于本实践需要特别注意两点：

- 论文中的 expert 使用局部 height scan，student 使用深度图；本作业不训练深度图 student，而是把 RayCaster height scan 直接接入当前策略与 critic。
- 论文给出的 expert height scan 与本作业的网格配置并不完全相同。请以作业代码中的要求为准，使用 `1.6 m × 1.6 m`、分辨率 `0.1 m` 的扫描网格。

2.3 数据来源

本实践使用的数据来自 [OmniRetarget Dataset](#)。该数据集提供 G1 人形机器人与物体、复杂地形交互的运动轨迹，由 OmniRetarget 生成。

数据集包含三个子集：

子集	内容	数据集页面标注时长
robot-object/	机器人搬运物体	3.0 小时
robot-terrain/	机器人攀爬复杂地形	0.5 小时
robot-object-terrain/	同时涉及物体和地形的交互	0.5 小时

本次作业使用 robot-terrain/ 中的攀爬数据。原始无物体轨迹的 .npz 文件主要包含：

- fps：轨迹帧率。
- qpos：形状为 [T, 36] 的状态序列。
- 前 7 维为浮动基座姿态与位置：[qw, qx, qy, qz, x, y, z]。
- 后 29 维为 G1 的关节位置。

当前仓库已经包含作业所需的数据及转换结果，默认使用：

代码块

```
1 datasets/robot-terrain/climb_15_z_scale_1.0.npz
2 datasets/hoi_mimic_data/climb_15_z_scale_1.0_mimic.npz
3 datasets/hoi_mimic_data/climb_15_z_scale_1.0_mimic.terrain.json
4 datasets/models/terrain/climb_15/multi_boxes_z_scale_1.0_isaac_world.urdf
```

因此，本作业**不要求**下载完整数据集，也不要求实现数据重定向、Motion Matching 或数据转换。仓库中的转换脚本仅用于教师准备数据或课后扩展。

3. 项目结构与关键文件

 下载本次作业的代码: https://pan.baidu.com/s/1fbSggWFaxc_mL-ZXyYMANg?pwd=nk8r

本次作业主要涉及以下目录：

代码块

```
1 HOI_Mimic/
2 |— set_project_root.sh
3 |— unitree_rl_lab.sh
4 |— scripts/
5 |   |— rsl_rl/
6 |   |   |— train.py           # PPO 训练入口
7 |   |   |— play.py           # checkpoint 回放与 RayCaster 调试入口
8 |   |— mimic/                # 数据转换/可视化工具，本作业通常不修改
9 |— datasets/
```

```

10 |   |— robot-terrain/                # OmniRetarget 原始 G1 terrain 轨迹
11 |   |— hoi_mimic_data/              # 已转换的 motion 与 terrain metadata
12 |   |— models/terrain/              # 地形 URDF
13 |   |— rsl_rl/                      # 实践使用的 PPO/RSL-RL 库
14 |   |— source/unitree_rl_lab/unitree_rl_lab/tasks/mimic/
15 |       |— mdp/
16 |       |   |— hoi_height_scan.py    # TODO 1: metadata 读取与校验
17 |       |   |— sensors/
18 |       |   |— hoi_merged_terrain_ray_caster.py    # 将各环境的 box 和 ground
plane 合并为 Warp mesh
19 |       |   |— hoi_merged_terrain_ray_caster_cfg.py    # 自定义 RayCaster 的配置类
型
20 |       |— robots/g1_29dof/
21 |           |— hoi_mimic_terrain/
22 |           |   |— tracking_env_cfg.py    # blind 基线任务, 提供 motion、scene、
reward、termination、curriculum 等基础配置
23 |           |   |— hoi_mimic_terrain_perceptive_raycast/
24 |           |   |— tracking_env_cfg.py    # TODO 2、3: 感知任务配置

```

4. 部分原理解释

4.1 HOI motion tracking

传统 velocity tracking 任务主要要求机器人跟踪速度指令，HOI motion tracking 则要求机器人在每个时刻跟踪一段完整的全身参考动作。

项目转换后的 mimic motion 数据包含以下信息：

代码块

```

1  fps
2  joint_pos
3  joint_vel
4  body_pos_w
5  body_quat_w
6  body_lin_vel_w
7  body_ang_vel_w

```

环境从 motion 中取得目标关节状态和目标刚体状态，并根据机器人当前状态与参考状态之间的误差计算奖励。当前 blind 基线的主要奖励包括：

- anchor 全局位置和姿态跟踪；
- 各关键刚体相对位置和姿态跟踪；
- 刚体线速度和角速度跟踪；

- 动作变化率、关节限位、部分关节力矩限制和非期望接触惩罚。

如果 anchor 高度误差、anchor 姿态误差或末端刚体高度误差超过阈值，episode 会提前终止。训练还使用 curriculum 逐步提高 push disturbance 与 domain randomization 强度。

4.2 terrain metadata 与 merged mesh

`*.terrain.json` 中的 `mjcf_boxes` 描述地形由哪些 box primitive 组成。每个 box 的核心字段为：

字段	含义	形状/顺序
<code>pos</code>	box 在 terrain body 局部坐标系中的中心位置	长度 3
<code>quat</code>	box 在 terrain body 局部坐标系中的姿态	<code>wxyz</code> ，长度 4
<code>half_size</code>	box 的半边长	长度 3
<code>full_size</code>	可选的完整边长	长度 3，需除以 2

自定义 `HoiMergedTerrainRayCaster` 在初始化时执行以下操作：

1. 找到每个并行环境中的 `HOI_Terrain` prim。
2. 读取一次 `mjcf_boxes`。
3. 将 box 从 terrain 局部坐标系转换到世界坐标系。
4. 为所有 box 生成三角网格。
5. 为每个环境加入 ground plane patch。
6. 合并为一个 Warp mesh，供后续批量 raycast 使用。

`source_npz` 等 metadata 字段只记录数据来源，不参与当前 RayCaster 加载；运行时真正需要的是 `terrain_init_pos`、`terrain_init_quat` 和 `mjcf_boxes`。

4.3 RayCaster 扫描网格

RayCaster 挂载在：

代码块

```
1 {ENV_REGEX_NS}/Robot/torso_link
```

使用的扫描参数为：

- `ray_alignment="yaw"`：网格跟随机器人平移和 yaw，但不跟随 roll/pitch 倾斜。

- `resolution=0.1`。
- `size=[1.6, 1.6]`。

网格两端都包含采样点，因此每个方向有：

代码块

```
1 1.6 / 0.1 + 1 = 17
```

单帧扫描点数量为：

代码块

```
1 17 × 17 = 289
```

使用 yaw 对齐而不是完整姿态对齐，可以让扫描网格始终保持水平，同时随机器人朝向转动，避免机器人俯仰或翻滚时扫描平面一起倾斜。

4.4 height scan observation

本项目使用 Isaac Lab locomotion 任务中的相对高度语义：

代码块

```
1 height = sensor_pos_w.z - ray_hit_w.z - offset
```

本作业要求 `offset=0.5`，并将输出裁剪到 `[-1.0, 5.0]`。

需要区分两个容易混淆的概念：

- RayCaster 的安装位置决定射线从哪里发出。
- observation term 的 `offset=0.5` 决定输出高度值的零点偏移。

对于 policy，height scan 还会加入 `[-0.02, 0.02]` 的均匀噪声，提高策略对感知误差的鲁棒性。critic 使用同样的高度定义和裁剪范围，但不添加该噪声，以获得更稳定的价值估计。

4.5 历史观测

blind 任务定义：

代码块

```
1 PROPRIO_HISTORY_LENGTH = 8
```

本作业中的 policy 和 critic height scan 都应使用相同的 `history_length`。Observation Manager 会负责保存和拼接历史帧；不要在 sensor 或 observation function 中再次手工维护 8 帧队列。

对 height scan 而言，单帧为 289 个值，历史长度为 8。历史信息使策略能够从连续高度变化中感知自己相对障碍物的运动趋势，而不只是观察当前瞬时地形。

5. 环境准备

5.1 基础环境

建议使用与项目匹配的环境：

- Ubuntu
- NVIDIA GPU 与可用的 CUDA 驱动
- Isaac Sim 5.1
- Isaac Lab 2.3.0
- 已安装 Isaac Lab 的 Python/conda 环境，例如 `env_isaaclab`
- 可选：Weights & Biases

激活环境：

代码块

```
1  conda activate env_isaaclab
```

可以使用TensorBoard或者W&B，如果使用 W&B：

代码块

```
1  pip install wandb
2  wandb login
```

5.2 安装实践使用的 rsl_rl 库

本仓库包含最新的 `rsl_rl` 库，默认 Isaac Lab 环境中的 PyPI 版本不包含项目修改。首次配置环境时执行：

代码块

```
1  cd /path/to/HOI_Mimic
2  pip install -e rsl_rl/
```

5.3 安装 unitree_rl_lab

代码块

```
1 cd /path/to/HOI_Mimic
2 ./unitree_rl_lab.sh -i
```

安装完成后重新激活 conda 环境，再回到项目根目录。

5.4 每个新 shell 的初始化

每次打开新 shell 后执行：

代码块

```
1 conda activate env_isaacclab
2 cd /path/to/HOI_Mimic
3 source set_project_root.sh
```

set_project_root.sh 会设置：

环境变量	默认值
PROJECT_ROOT	当前仓库根目录
HOI_ROOT	\$PROJECT_ROOT/datasets
HOI_MIMIC_DATA_DIR	\$PROJECT_ROOT/datasets/hoi_mimic_data
HOI_MIMIC_TERRAIN_MOTION_FILE	climb_15_z_scale_1.0_mimic.npz
HOI_MIMIC_TERRAIN_META_FILE	climb_15_z_scale_1.0_mimic.terrain.json
HOI_MIMIC_TERRAIN_URDF	multi_boxes_z_scale_1.0_isaac_world.urdf

感知任务默认需要 *_isaac_world.urdf。如果之前为了 blind 任务手工修改过 HOI_MIMIC_TERRAIN_URDF，建议打开新 shell，或清除旧变量后重新执行脚本。

6. 运行前检查

6.1 检查任务注册

代码块

```
1 ./unitree_rl_lab.sh -l | grep -E '\\| Unitree-G1-29dof-Mimic-  
HOI_terrain[:space:]]+\\|\\| Unitree-G1-29dof-Mimic-HOI_terrain-Perceptive-  
Raycast[:space:]]+\\|'
```

应至少看到两个任务ID：

- Unitree-G1-29dof-Mimic-HOI_terrain
- Unitree-G1-29dof-Mimic-HOI_terrain-Perceptive-Raycast

6.2 检查默认数据

代码块

```
1 test -f "${HOI_MIMIC_TERRAIN_MOTION_FILE}" && echo "motion OK" || echo "motion  
MISSING"  
2 test -f "${HOI_MIMIC_TERRAIN_META_FILE}" && echo "metadata OK" || echo  
"metadata MISSING"  
3 test -f "${HOI_MIMIC_TERRAIN_URDF}" && echo "URDF OK" || echo "URDF MISSING"
```

检查 metadata 是否包含非空 `mjcf_boxes`：

代码块

```
1 python3 -c "import json, os;  
d=json.load(open(os.environ['HOI_MIMIC_TERRAIN_META_FILE'])); assert  
isinstance(d.get('mjcf_boxes'), list) and d['mjcf_boxes']; print('mjcf_boxes  
OK:', len(d['mjcf_boxes']))"
```

检查 URDF 是否是感知任务所需版本，预期文件为

`multi_boxes_z_scale_1.0_isaac_world.urdf`：

代码块

```
1 echo "${HOI_MIMIC_TERRAIN_URDF}"
```

7. 作业核心任务

本次作业只要求完成三组核心 TODO。

7.1 TODO 1：读取并校验 terrain metadata

文件：

需要补全函数：

代码块

```
1 _normalize_box_record(entry, idx)
2 _load_boxes(metadata_file, device)
```

- `_normalize_box_record()` 函数负责校验一个 box record，并返回规范化的 `pos`、`quat` 和 `half_size`。实现要求：
 - 从 `entry` 读取 `pos`、`quat` 和 `half_size`。
 - 如果没有 `half_size`，但存在 `full_size`，则将每个完整边长除以 2。
 - 校验 `pos` 长度为 3。
 - 校验 `quat` 长度为 4，顺序保持为 `wxyz`。
 - 校验 `half_size` 长度为 3。

代码块

```
1 pos:          长度 3
2 quat:         长度 4, wxyz
3 6. 数据非法时抛出 `ValueError`，错误信息中必须包含 box 下标 `idx`，便于定位
   metadata 中的具体记录。
4
5 返回值形式：
6
7 ```text
8 pos:          长度 3
9 quat:         长度 4, wxyz
10 half_size: 长度 3
```

- `_load_boxes()` 函数实现要求：
 - 使用 UTF-8 打开 `metadata_file` 并读取 JSON。
 - 读取 `mjcf_boxes`。
 - 确认 `mjcf_boxes` 是非空 list。
 - 对每个 box 调用 `_normalize_box_record()`。
 - 将结果转换为 `torch.float32` tensor。
 - tensor 必须位于调用方传入的 `device`。

返回 tensor 的形状必须为：

代码块

```
1 box_pos_local: [num_boxes, 3]
2 box_quat_local: [num_boxes, 4]
3 box_half:      [num_boxes, 3]
```

JSON 不应在每个仿真 step 中重复读取。当前 RayCaster 会在 mesh bake 阶段调用 loader；也可以按 (metadata_file, device) 实现缓存，但不要把逐帧文件 I/O 引入 observation 计算。

hoi_height_scan.py 中的 _build_grid_xy()、compute_hoi_height_scan()、hoi_height_scan() 不在本次作业范围内，不需要实现。

7.2 TODO 2：配置 RayCaster height scanner

文件：

代码块

```
1 source/unitree_rl_lab/unitree_rl_lab/tasks/mimic/robots/g1_29dof/
2 └─ hoi_mimic_terrain_perceptive_raycast/tracking_env_cfg.py
```

将 RobotSceneCfg.height_scanner 从 None 替换为 HoiMergedTerrainRayCasterCfg。

配置项	要求	原因
prim_path	{ENV_REGEX_NS}/Robot/torso_link	扫描器随 torso 移动
ray_alignment	"yaw"	网格水平，仅跟随机器人朝向
pattern_cfg	GridPatternCfg(resolution=0.1, size=[1.6, 1.6])	生成 17 × 17 扫描点
terrain_prim_path	每个环境中的 HOI_Terrain	找到需要烘焙的地形 prim
metadata_file	blind_cfg.TERRAIN_META_FILE	与 blind task 使用同一地形 metadata
use_mjcf_boxes_mesh	True	使用 mjcf_boxes 构建 mesh
include_ground_plane	True	避免 box 之外的射线全部 miss

rebake_on_reset

False

默认地形位姿固定，无需每次 reset 重建 mesh

7.3 TODO 3: 接入 policy 与 critic observation

文件：

代码块

```
1 source/unitree_rl_lab/unitree_rl_lab/tasks/mimic/robots/g1_29dof/  
2   └─ hoi_mimic_terrain_perceptive_raycast/tracking_env_cfg.py
```

需要分别补全 `ObservationsCfg.PolicyCfg.height_scanner` 和 `ObservationsCfg.PrivilegedCfg.height_scanner`

两者共同要求：

代码块

```
1 func          = mdp.height_scan  
2 sensor_cfg    = SceneEntityCfg("height_scanner")  
3 offset        = 0.5  
4 clip          = (-1.0, 5.0)  
5 history_length = blind_cfg.PROPRIO_HISTORY_LENGTH
```

policy 额外加入噪声：

代码块

```
1 Unoise(n_min=-0.02, n_max=0.02)
```

critic 不添加该噪声。

8. 训练与验收

建议按照“blind 基线—感知 smoke train—可视化—正式训练—checkpoint 回放”的顺序验证。这样出现问题时，可以区分基础环境问题和 RayCaster 作业实现问题。

8.1 可选：blind 基线 smoke train

代码块

```
1 cd "${PROJECT_ROOT}"  
2 conda activate env_isaacrlab
```

```
3 source set_project_root.sh
4
5 python scripts/rsl_rl/train.py \
6     --task Unitree-G1-29dof-Mimic-HOI_terrain \
7     --num_envs 64 \
8     --max_iterations 10 \
9     --headless \
10    --logger tensorboard
```

如果 blind 任务无法创建，优先检查 Isaac Lab 安装、任务注册、数据路径和实践用到的 `rsl_rl`，不要先调试 RayCaster TODO。

8.2 必做：感知任务 smoke train

代码块

```
1 cd "${PROJECT_ROOT}"
2 source set_project_root.sh
3
4 python scripts/rsl_rl/train.py \
5     --task Unitree-G1-29dof-Mimic-HOI_terrain-Perceptive-Raycast \
6     --num_envs 4 \
7     --max_iterations 10 \
8     --headless \
9     --logger tensorboard
```

验收要求：

- 环境成功创建。
- 能看到 merged HOI RayCaster mesh bake 的日志。
- observation shape 正常。
- 完成 10 次 iteration。
- 不出现路径错误、shape 错误、作业 TODO 的 `NotImplementedError` 或 `NaN/Inf`。
- 训练结束后生成最终 checkpoint。

smoke train 的目标是验证链路，不代表策略已经收敛。

8.3 RayCaster 可视化与数值检查

使用 smoke train 或后续训练得到的 checkpoint：

代码块

```
1 python scripts/rsl_rl/play.py \
```

```
2  --task Unitree-G1-29dof-Mimic-HOI_terrain-Perceptive-Raycast \  
3  --checkpoint <checkpoint_path> \  
4  --num_envs 1 \  
5  --hoi-play-no-curriculum \  
6  --hoi-play-no-dr \  
7  --height-scan-vis \  
8  --height-scan-print
```

可视化验收要求：

- 扫描网格位于 torso 周围。
- 网格随机器人位置移动。
- 机器人转向时网格随 yaw 转动。
- 机器人 roll/pitch 变化时网格仍保持水平。
- 命中点位于 ground plane 或 box 表面附近。
- 扫描高度的 min/max/mean 等统计为有限值。
- 射线在 box 之外仍可命中 ground plane，不应大面积出现 `inf`。

8.4 正式训练

参考训练命令：

代码块

```
1  python scripts/rsl_rl/train.py \  
2  --task Unitree-G1-29dof-Mimic-HOI_terrain-Perceptive-Raycast \  
3  --num_envs 4096 \  
4  --max_iterations 50000 \  
5  --headless \  
6  --logger wandb \  
7  --log_project_name unitree_hoi_mimic_terrain_perceptive_raycast
```

不用 W&B 时：

代码块

```
1  python scripts/rsl_rl/train.py \  
2  --task Unitree-G1-29dof-Mimic-HOI_terrain-Perceptive-Raycast \  
3  --num_envs 1024 \  
4  --max_iterations 50000 \  
5  --headless \  
6  --logger tensorboard
```

4096个环境和 50000 次 iteration 是参考配置，学生可根据课程算力调整。如果 CUDA OOM，依次尝试减少到 2048、1024 或更低。

训练日志通常位于：

代码块

```
1 logs/rs1_rl/unitree_g1_29dof_mimic_hoi_terrain_perceptive_raycast/  
2   └─ <timestamp>/  
3     └─ model_<iteration>.pt  
4     └─ params/
```

使用 TensorBoard 查看训练曲线：

代码块

```
1 tensorboard \  
2   --logdir logs/rs1_rl/unitree_g1_29dof_mimic_hoi_terrain_perceptive_raycast \  
3   --port 6006
```

8.5 回放 checkpoint

代码块

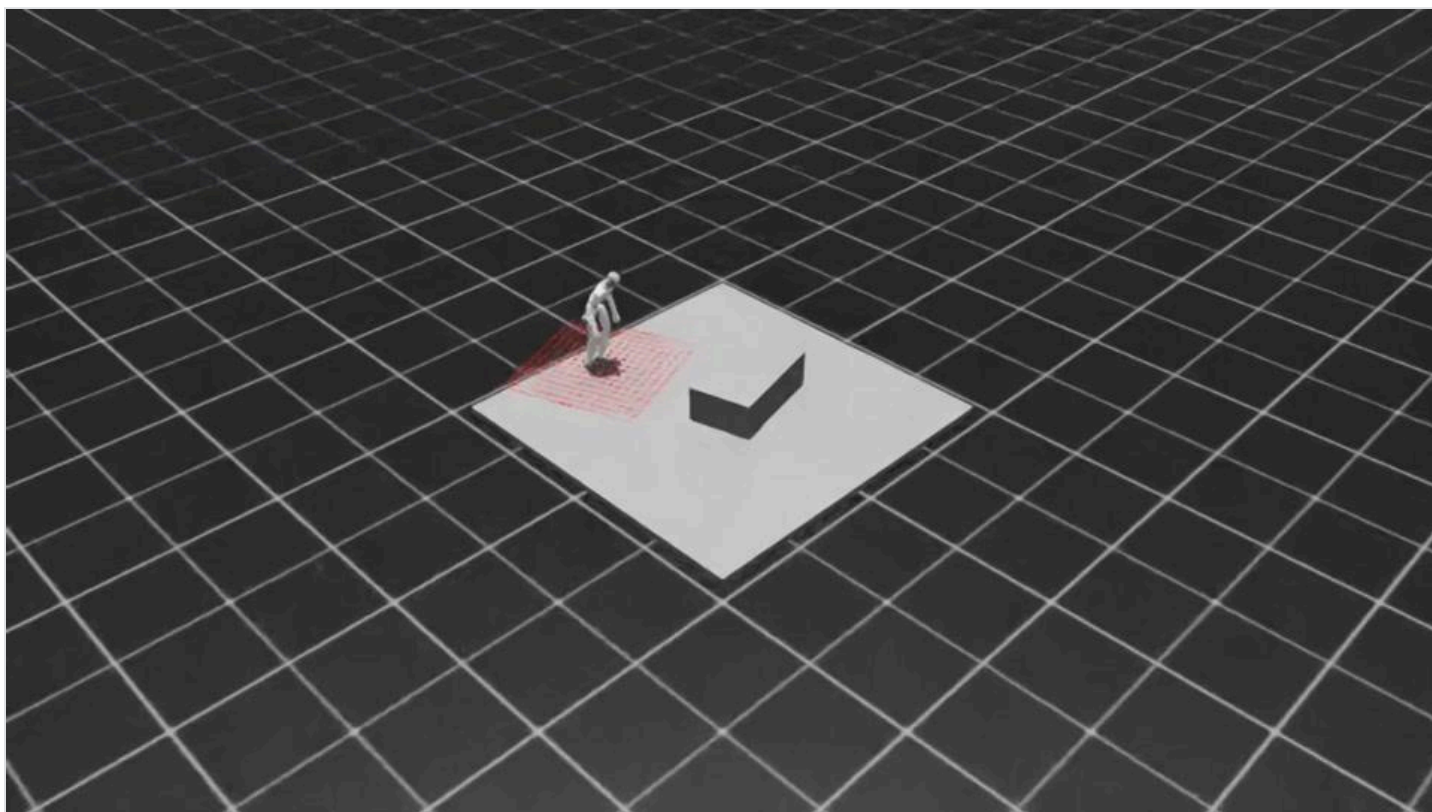
```
1 python scripts/rs1_rl/play.py \  
2   --task Unitree-G1-29dof-Mimic-HOI_terrain-Perceptive-Raycast \  
3   --checkpoint <checkpoint_path> \  
4   --num_envs 1 \  
5   --hoi-play-no-curriculum \  
6   --hoi-play-no-dr \  
7   --height-scan-vis \  
8   --height-scan-print
```

常用回放参数：

参数	作用
<code>--hoi-play-no-curriculum</code>	回放时关闭 curriculum level-up
<code>--hoi-play-no-dr</code>	冻结 startup domain randomization
<code>--hoi-play-no-obs-noise</code>	关闭 policy observation corruption
<code>--hoi-play-exact-motion-joints</code>	reset 时精确对齐参考关节，不加 joint resample noise

<code>--hoi-play-deterministic</code>	同时关闭 curriculum、DR 和 observation noise
<code>--height-scan-vis</code>	显示 RayCaster 射线/命中点
<code>--height-scan-print</code>	打印 height scan 统计

建议首先只使用 `--hoi-play-no-curriculum --hoi-play-no-dr`。训练时 policy 使用 observation noise，直接关闭 noise 会改变观测分布，因此 `--hoi-play-no-obs-noise` 不一定让表现更好。



9. 提交要求与评分标准

9.1 提交内容

请提交：

1. 补全后的 `hoi_height_scan.py`
2. 补全后的 RayCaster `tracking_env_cfg.py`
3. 感知任务 smoke train 的终端输出
4. RayCaster 命中点截图或录屏
5. 正式训练日志、checkpoint 路径与回放截图/视频

如果提交内容为全部项目，请同时提交 `git diff --stat`，用于说明代码改动范围

9.2 评分标准

项目	分值	验收重点
TODO 1: metadata loader	20	数据读取、字段校验、full_size 转换、shape、dtype、device 正确
TODO 2: RayCaster 配置	20	挂载、扫描网格、地形 mesh 和 reset 策略正确
TODO 3: observation 接入	20	policy/critic 的 height 语义、history、noise 和 clipping 正确
Smoke train 与数值检查	10	能连续运行且无 NaN/Inf
RayCaster 可视化	10	扫描点、网格运动方式与高度统计合理
训练与回放	15	生成 checkpoint 并成功播放，策略具有合理的 motion tracking 表现
代码质量与改动范围	5	实现清晰、没有无关改动